

Boot Sequence

Contents

Boot Process

Syscon

First Loader

Second Loader

Secure Kernel

Secure Kernel Bootloader

Non-secure Kernel Bootloader

ScePsp2BootConfig

FW 0.931.010 special case

SceSysStateMgr

Boot Partition

Boot Debug Checkpoint Codes

GPIO

Known Codes

Suspend and Resume

Boot Process

Syscon

Syscon powers up and sets up DRAM, sets up boot context buffer, turns on the KERMIT SOC which eventually starts the Boot code on the CMeP Processor.

First Loader

The PS Vita main application processor is an ARM Cortex A9 MPCore (http://infocenter.arm.com/help/topic/com.arm.doc.ddio407i/DDIo407I_cortex_a9_mpcore_r4p1_trm.pdf). It implements ARM TrustZone for execution in both a non-secure world and a sandboxed Secure World. However it is not the first processor to run on boot.

The cmep processor is the actual secure boot device rather than the ARM processor. The cmep processor's boot ROM, nicknamed First Loader, is the first known code running on PS Vita start. Once it starts, it likely maps the eMMC and directly reads in the second_loader.enp from the eMMC (SLB2 partition) or SD Card (if SD BOOT challenge passes). This is in the native load format of the boot

ROM. There are two layers of encryption. First it decrypts the per-console SLSK personalization layer that was added during the System Software installation. After that, it decrypts the factory-encrypted SLSK layer then begins execution.

Second Loader

The Second Loader is primarily responsible for preparing the ARM processor. It initializes DRAM and decrypts `kernel_boot_loader.self` from eMMC SLB2 partition into DRAM. It also writes the ARM exception vector and some boot context information to the 32kB scratch buffer (mirror mapped to `0x00000000` on ARM). `kernel_boot_loader.self` contains both the secure kernel bootloader and TrustZone, as well as the non-secure kernel bootloader. At this point the `kprx_auth_sm.self` and `prog_rvk.srvk` read from the eMMC SLB2 partition are both loaded into DRAM. Finally, the Second Loader resets itself with a pointer to the `secure_kernel.enp` binary.

Secure Kernel

the cmep processor then restarts and loads the `secure_kernel.enp` in and again decrypts the per-console layer that was added during the System Software installation, and the factory layer. At this point the cmep processor is prepared and Secure Kernel tells Syscon to reset the ARM CPU at `0x00000000` (cmep scratch buffer). This triggers the ARM secure boot process.

Secure Kernel Bootloader

The secure kernel bootloader decompresses the ARZL compressed TrustZone kernel, loads it and sets up the VBAR and MVBAR. It then decompresses the ARZL non-secure kernel bootloader, sets NS in SCR and jumps into non-secure kernel bootloader with svc mode. See Kernel Boot Loader for more information.

Non-secure Kernel Bootloader

The non-secure kernel bootloader contains an embedded and likely stripped version of SceSystemem, SceKernelModulemgr, SceSblSmschedProxy, and some other core drivers. The NSKBL sets up the eMMC device (again) and starts `os0:psp2bootconfig.skprx`.

ScePsp2BootConfig

This kernel module does not export any library. It only has a module init function that has a hard coded list of core kernel modules (ex: `system.skprx`) which are loaded with calls back into NSKBL through `SceKblForKernel` imports. Once the core initialization is done, the next module to run is `SceSysStateMgr`.

FW 0.931.010 special case

On System Software version 0.931.010 (and probably earlier versions), NSKBL embeds the kernel modules list in data segment instead of using the ScePsp2BootConfig kernel module. Here is the list:

```
system.skprx
excpmgr.skprx
intrmgr.skprx
systemer.skprx
acmgr.skprx
threadmgr.skprx
dmacmgr.skprx
ssproxy.skprx
smc_proxy.skprx
authmgr.skprx
iofilemgr.skprx
modulemgr.skprx
processmgr.skprx
backtrace.skprx
sdbgsdio.skprx
dec4p_sdfmgr.skprx
dec4p_sttyp.skprx
dec4p_sdbgp.skprx
dec4p_sdrfp.skprx
stdio.skprx
lowio.skprx
clockgen.skprx
sdif.skprx
sdstor.skprx
fatsd.skprx
exfatfs.skprx
pamgr.skprx
sysstatemgr.skprx
```

SceSysStateMgr

This kernel module also does not export any library. Its init function first maps all the [SceKernelBootimage](#) embedded modules and redirects them to `oso:kd/`. Then it decrypts `os0:psp2config.skprx` or `os0:psp2config_vita.skprx` or `os0:psp2config_dolce.skprx` and parses the [System Configuration Script](#) to load the remaining modules and finally either [SceSafeMode](#) or [SceShell](#) or [ScePsp2Swu](#) or [ScePsp2Diag](#).

Also refer to the [SceSysStateMgr](#) page for System Configuration Script.

Boot Partition

The boot partition, named `sloader`, is [SLB2](#) formatted.

Boot Debug Checkpoint Codes

During the boot sequence, the various bootloaders will update a GPIO register specifying the progress into boot. This can be used to debug where in the boot process something fails.

Second Loader checkpoint codes start at 0x40 (e.g. GPO value 0x52 corresponds to SBL code 0x12).

GPIO

The GPIO registers are registered at 0xE20A000C (turn off bits) and 0xE20A0008 (turn on bits). On PDEL units, this maps to the LED lights.

Known Codes

The *Event* column indicates what happens/is about to happen when a code is shown on the GPO LED. If boot of the unit doesn't succeed, the *Halting event* column indicates what caused the boot process to fail based on the last value of the GPO LED.

Code	Location	Event	Halting event
64 / 0x40	second_loader (0.931.010)	SBL finished successfully	
65 / 0x41	second_loader	Some Hardware Info check complete - GPO initialization OK?	
66 / 0x42	second_loader (0.931.010)	Something with Syscon	ERROR: communication with Syscon failed
67 / 0x43	second_loader	Register bigmac key 0x508 and 0x51B complete	ERROR: communication with Syscon failed
68 / 0x44	second_loader		ERROR: ?some check with keyring 0x501 and Cmp data? failed
69 / 0x45	second_loader	?Initializing LPDDR2?	ERROR: ?LPDDR2 initialization failed?
70 / 0x46	second_loader	Setting QA flags to bigmac keyring complete	
71 / 0x47	second_loader (0.931.010)	Calling testMemory	?ERROR: testMemory failed?
72 / 0x48	second_loader (0.931.010)	?Initializing SD/eMMC?	ERROR: ?SD/eMMC initialization failed?
73 / 0x49	second_loader (0.931.010)	?Loading kernel_boot_loader.self?	ERROR: ?reading/loading/... kernel_boot_loader.self failed?
74 / 0x4A	second_loader (0.931.010)		ERROR: ?SD/eMMC I/O error?
75 / 0x4B	second_loader (0.931.010)		WARNING: Reading ConsoleID from eMMC failed
76 / 0x4C	second_loader (0.931.010)		ERROR: ?starting ARM clock failed? (cannot happen on System Software version 0.931.010)
77 / 0x4D	second_loader (0.931.010)	?Initializing SNVS?	ERROR: Writing 0x502-0x504/0x50B/... keyring failed (?SNVS init failed?) (cannot happen on System Software version 0.931.010)
78 / 0x4E	second_loader (0.931.010)		ERROR: SBL version mismatch with SVNS-stored System Software version
79 / 0x4F	second_loader (0.931.010)		ERROR: testMemory failed (dummy in System Software version 0.931.010, cannot happen)
80 / 0x50	second_loader (0.931.010)	Copying keyrings 0x602/0x601 to physical address 0xE0020100/0xE0020200	
82 / 0x52	second_loader		ERROR: eMMC is not available
83 / 0x53	second_loader		WARNING: Reading OpenPSID from eMMC failed
84 / 0x54	second_loader		ERROR: Minimal System Software version read failed / SBL version is lower than minimal System Software version

85 / 0x55	second_loader	Setting factory System Software version to Bigmac keyring complete	WARNING: Something related to Syscon communication failed?
86 / 0x56	second_loader		
87 / 0x57	second_loader (0.931.010)		WARNING: ???
88 / 0x58	second_loader (0.940)		WARNING: invalid/mismatched per-console information?
89 / 0x59	second_loader (0.931.010)		WARNING: ???
90 / 0x5A	second_loader	About to write SceKblParam to SPAD32K	
94 / 0x5E	second_loader (0.931.010)		ERROR: SVNS-stored System Software version is lower than minimal System Software version
96 / 0x60	second_loader	Setting SceKblParam complete and Start setting some device clock.	
129 / 0x81	Secure Kernel BootLoader	Core 0 (secure world) pre-init complete	
130 / 0x82	Secure Kernel BootLoader	Secure world interrupts registered (?)	
131 / 0x83	Secure Kernel BootLoader	Serial console ready, boot message printed	
132 / 0x84	Secure Kernel BootLoader	Some device init	
133 / 0x85	Secure Kernel BootLoader	Some co-processor init. Starting point for other cores.	
134 / 0x86	Secure Kernel BootLoader	MMU enabled, VBAR/MVBAR set up	
135 / 0x87	Secure Kernel BootLoader	Nothing since 134	
136 / 0x88	Secure Kernel BootLoader	Boot setup complete, secure kernel loading begin	
137 / 0x89	Secure Kernel BootLoader	Secure kernel loaded. About to load NS KBL at 0x51000000	
138 / 0x8A	Secure Kernel BootLoader	Secure kernel loaded. About to resume context at 0x1F000000.	ERROR: Undefined Instruction exception
139 / 0x8B	Secure Kernel BootLoader		ERROR: SVC exception (should not happen)
140 / 0x8C	Secure Kernel BootLoader		ERROR: Prefetch abort exception
141 / 0x8D	Secure Kernel BootLoader		ERROR: Data abort exception
142 / 0x8E	Secure Kernel BootLoader		ERROR: IRQ exception (should not happen)
143 / 0x8F	Secure Kernel BootLoader		ERROR: FIQ exception (should not happen)

161 / 0xA1	Non-Secure Kernel BootLoader	Core 0 (non-secure world) pre-init complete	
162 / 0xA2	Non-Secure Kernel BootLoader	Some interrupts registered (?)	
163 / 0xA3	Non-Secure Kernel BootLoader	Serial console ready, boot message printed (if enabled)	
164 / 0xA4	Non-Secure Kernel BootLoader	Some buffer is initialized to device addresses	
165 / 0xA5	Non-Secure Kernel BootLoader	Some co-processor init. Starting point for other cores.	
166 / 0xA6	Non-Secure Kernel BootLoader	MMU enabled, VBAR set up	
167 / 0xA7	Non-Secure Kernel BootLoader	Nothing since 166	
168 / 0xA8	Non-Secure Kernel BootLoader	Boot setup complete, NS kernel loading begin	
169 / 0xA9	Non-Secure Kernel BootLoader	Kernel pre-init (setup stacks, interrupts, etc) done. Right before first external loading.	
170 / 0xAA	Non-Secure Kernel BootLoader		ERROR: Undefined Instruction exception
171 / 0xAB	Non-Secure Kernel BootLoader		ERROR: SVC exception (should not happen)
172 / 0xAC	Non-Secure Kernel BootLoader		ERROR: Prefetch abort exception
173 / 0xAD	Non-Secure Kernel BootLoader		ERROR: Data abort exception
174 / 0xAE	Non-Secure Kernel BootLoader		ERROR: IRQ exception (should not happen)
175 / 0xAF	Non-Secure Kernel BootLoader		ERROR: FIQ exception (should not happen)

Suspend and Resume

Upon suspension, context is written to memory and a syscon command is issued to save the context pointer as well as other information (for example, if it should restart into update mode). When resuming, the boot process is the same as cold boot up until the secure kernel bootloader. After secure

kernel loads, instead of decompressing and jumping to the non-secure kernel bootloader, it restores the saved context and returns to the kernel resume code.

See also Suspend.

Retrieved from "http://wiki.henkaku.xyz/index.php?title=Boot_Sequence&oldid=21852"

This page was last edited on 15 August 2026, at 11:53.

Content is available under Creative Commons Attribution unless otherwise noted.